UD1.4. JavaScript, DOM y AJAX....2
Creación de un Catálogo Dinámico....2
I. Fundamentos Teóricos: La Capa de Comportamiento....2
1. JavaScript y el DOM (Document Object Model)....2
2. AJAX, Fetch y el Salto Dinámico....3
II. Desarrollo del Proyecto (Paso a Paso)....3
Paso 1: Configuración de micatalogo.html....3
Paso 2: Adaptación de estilos.css....4
Paso 3: El Motor Dinámico micatalogo.js....4
III. Rúbrica de Evaluación....6

UD1.4. JavaScript, DOM y AJAX

Creación de un Catálogo Dinámico

Objetivo: Integrar todas las tecnologías vistas (HTML, CSS, XML) usando **JavaScript** para transformar un catálogo estático en una aplicación web interactiva y que carga sus datos de forma dinámica.

Resultado de Aprendizaje (RA)	Criterios de Evaluación (CE)
RA6: Realiza la publicación de información, analizando las técnicas de programación en el lado cliente.	**CE 1.4:** Aplicar técnicas de programación en el lado cliente para la manipulación y presentación de información.
	CE 1.5: Crear o modificar componentes dinámicos de una página web para mejorar su interactividad y usabilidad.

I. Fundamentos Teóricos: La Capa de Comportamiento

1. JavaScript y el DOM (Document Object Model)

JavaScript es el lenguaje de programación del lado del cliente que dota de interactividad a las páginas web. Su papel principal en LMSGI es manipular el **DOM**.
El **DOM** es una interfaz de programación para documentos HTML y XML. El navegador web transforma tu código HTML en un **árbol de nodos** . JS utiliza el DOM para:

- **Seleccionar** cualquier nodo (ej: un <h1> o un).
- **Manipular** sus atributos, contenido o estilo.
- **Generar** nuevos nodos HTML desde cero.

Concepto	Código Clave	Función
Selección por ID	document.getElementById('id-elemento')	Función fundamental para localizar un elemento.
Manipulación	elemento.textContent = 'Nuevo Texto'	Cambia el texto visible de un nodo.
Eventos	elemento.addEventListener('click', funcion)	Ejecuta una función cuando ocurre una acción (ej: un clic).
Manipulación CSS	elemento.classList.toggle('clase')	Alterna una clase CSS (ej: oculto) para cambiar el estilo.

2. AJAX, Fetch y el Salto Dinámico

AJAX (Asynchronous JavaScript and XML) es el concepto que permite a JavaScript realizar peticiones de datos al servidor (o archivos locales) **sin recargar la página**.

- La función moderna para realizar peticiones AJAX es **fetch()**.
- Utilizamos fetch() para leer el contenido del archivo micatalogo.xml.
- El objetivo final es la **Generación Dinámica**: Recorrer los datos del XML y, por cada <producto>, construir e inyectar sus etiquetas HTML (<section>, <h2>, etc.) directamente en el <main> de la página.

II. Desarrollo del Proyecto (Paso a Paso)

Debemos modificar tres archivos: micatalogo.html, estilos.css y crear micatalogo.js.

Paso 1: Configuración de micatalogo.html

El archivo HTML se convierte en el **esqueleto** del proyecto. Es crucial eliminar el contenido estático de los productos.

Sección	Modificación Clave	Código Clave
Header	Añadir un id para la manipulación inicial.	<h1 id="titulo-catalogo">...</h1>
Main	Dejar completamente **vacío** y añadir un id. Los productos se inyectarán aquí.	<main id="catalogo-container"></main>
Cierre	Enlazar el script al final del <body> (mejora el rendimiento).	<script src="micatalogo.js"></script>

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Mi Catálogo - UD 1.4 Dinámico</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <header>
    <h1 id="titulo-catalogo"> Catálogo de Productos Tecnológicos </h1>
  </header>

  <main id="catalogo-container">
    </main>

  <footer>
    <p>&copy; 2025 LMSGI</p>
  </footer>

  <script src="micatalogo.js"></script>
</body>
</html>
```

Paso 2: Adaptación de estilos.css

Necesitamos una clase CSS que JavaScript pueda activar o desactivar para controlar la visibilidad de los detalles técnicos.

CSS

```
/* ... [Contenido existente de estilos.css] ... */

/* CLASE NECESARIA PARA JAVASCRIPT (TOGGLE) */
.oculto {
  /* La regla clave para esconder un elemento */
  display: none;
}

.detalles-extra {
  /* Estilos de presentación para los detalles (opcional) */
  margin-top: 10px;
  padding: 10px;
  border-left: 3px solid #00bcd4;
  background-color: #f0ffff;
}
```

Paso 3: El Motor Dinámico micatalogo.js

Este archivo implementa la lógica completa:

JavaScript

```
// Archivo: micatalogo.js

document.addEventListener('DOMContentLoaded', function() {

  const tituloPrincipal = document.getElementById('titulo-catalogo');
  const mainContainer = document.getElementById('catalogo-container');

  // 1. MANIPULACIÓN INICIAL: Confirmar que JS está activo
  if (tituloPrincipal) {
    tituloPrincipal.textContent = 'Catálogo Dinámico (Cargando datos XML...)';
  }

  function cargarYGenerarCatalogo() {

    // 2. FETCH/AJAX: Solicitar el archivo XML
    fetch('micatalogo.xml')
      .then(response => response.text()) // Convertir la respuesta a texto plano
      .then(xmlTexto => {

        // 3. PARSEO XML: Convertir el texto a un objeto manipulable
        const parser = new DOMParser();
        const xmlDoc = parser.parseFromString(xmlTexto, "application/xml");

        // Actualizar el título al tener los datos listos
        if (tituloPrincipal) {
```

```
    tituloPrincipal.textContent = 'Catálogo de Productos - Interactivo LMSGI';
}

// 4. RECORRIDO Y GENERACIÓN DINÁMICA
const productosXML = xmlDoc.querySelectorAll('producto');

productosXML.forEach((productoXML, index) => {

    // a) Extracción de datos (Lectura de nodos y atributos)
    const nombre = productoXML.querySelector('nombre').textContent;
    const descripcion = productoXML.querySelector('descripcion').textContent;
    const precio = productoXML.querySelector('precio').textContent;
    const moneda = productoXML.querySelector('precio').getAttribute('moneda');
    const categoria = productoXML.querySelector('categoria').textContent;
    const imagenUrl = productoXML.querySelector('imagen').getAttribute('url');
    const idProducto = productoXML.getAttribute('id');

    // b) CREACIÓN DE LA ESTRUCTURA HTML (se usa innerHTML para simplificar)
    const section = document.createElement('section');
    section.classList.add('producto-destacado');

    section.innerHTML = `
        <img src="${imagenUrl}" alt="${nombre}" class="producto-img">
        <h2>${nombre}</h2>
        <p>${descripcion}</p>
        <p class="precio">Precio: ${precio} ${moneda}</p>
        <a href="#" class="enlace-compra">Comprar ahora</a>

        <button id="btn-detalles-${index}">Ver Detalles Técnicos</button>
        <div id="detalles-${index}" class="detalles-extra oculto">
            <p><strong>Referencia:</strong> ${idProducto}</p>
            <p><strong>Categoría:</strong> ${categoria}</p>
        </div>
    `;

    // c) INYECCIÓN en el DOM
    mainContainer.appendChild(section);

    // 5. ASIGNACIÓN DE EVENTOS (Interactividad)

    const botonDetalles = document.getElementById(`btn-detalles-${index}`);
    const detallesDiv = document.getElementById(`detalles-${index}`);

    if (botonDetalles && detallesDiv) {
        botonDetalles.addEventListener('click', function(e) {
            e.preventDefault();

            // CLAVE: Alternar la clase 'oculto'
            detallesDiv.classList.toggle('oculto');
```

```
                // Lógica de cambio de texto del botón
                if (detallesDiv.classList.contains('oculto')) {
                    this.textContent = 'Ver Detalles Técnicos';
                } else {
                    this.textContent = 'Ocultar Detalles';
                }
            });
        }
    });

})
.catch(error => {
    // Manejo de errores de carga del XML
    if (tituloPrincipal) {
        tituloPrincipal.textContent = 'ERROR: No se pudo cargar el catálogo. Revise la consola.';
    }
    console.error('Error en la carga del catálogo XML:', error);
});
}

// Iniciar el proceso
cargarYGenerarCatalogo();
});
```

III. Rúbrica de Evaluación

Puntuación	Descripción del Desempeño (CE 1.4 y CE 1.5)
2.5	El código JS está conectado y logra la **Selección básica** (getElementById) del <h1>. El intento de Fetch es nulo o el código de generación es incorrecto.
5	El código es funcional para la **Manipulación básica** (textContent) y demuestra comprensión de la sintaxis JS/DOM. Se implementa correctamente la estructura de fetch() y el parseo del XML.
7.5	El proyecto implementa **Fetch/AJAX** y logra **generar dinámicamente** todo el contenido de los productos (Título, Imagen, Precio) a partir del micatalogo.xml. El código para la interacción (click/toggle) está presente, pero puede ser incompleto o contener errores menores.
10	**Dominio de la Integración Dinámica:** El código es **funcional y completo**. Genera dinámicamente **TODO el HTML** a partir del XML y **cada producto generado es interactivo** con el efecto **Mostrar/Ocultar (Toggle)** y el cambio dinámico del texto del botón.